HUMBER POLYTECHNIC

Faculty of Applied Science and Technology

Lab #6 – Branches and Loops in Matlab

Michaël McCorkell

Dr. Maryam Davoudpour

November 12, 2025

Procedures and results:

1.

```
% Problem 8
% Modify elements of a user-input vector based on sign
clc; clear;

% Ask user to enter a vector
vec = input('Enter a vector of integers (e.g., randi([-10 20],1,19)): ');

% Display original vector
disp('Original vector:')
disp(vec)

% Initialize modified vector
modified = vec;

% Process each element
for i = 1:length(vec)
    if vec(i) > 0
        modified(i) = 2 * vec(i);
    elseif vec(i) < 0
        modified(i) = 3 * vec(i);
    end
end

% Display results
disp('Modified vector:')
disp(modified)
```

```
Enter a vector of integers (e.g., randi([-10 20],1,19)): randi([-10 20],1,19)
Original vector:
  Columns 1 through 17

    15    18    -7    18     9    -7    -2     6    19    19    -6    20    19     5    14    -6     3

  Columns 18 through 19

    18    14

Modified vector:
  Columns 1 through 17

    30    36   -21    36    18   -21    -6    12    38    38   -18    40    38    10    28   -18     6

  Columns 18 through 19

    36    28

>>
```

2.

```
% Problem 11
% Create n x n Pascal triangle matrix
clc; clear;
n = input('Enter number of rows for Pascal triangle: ');
% Initialize n x n matrix
C = zeros(n);
% Compute Pascal elements
for i = 1:n
    for j = 1:i
        C(i,j) = factorial(i-1) / (factorial(j-1) * factorial(i-j));
    end
end
disp('Pascal Triangle Matrix:')
disp(C)
```

```
Enter number of rows for Pascal triangle: 6
Pascal Triangle Matrix:
     1     0     0     0     0     0
     1     1     0     0     0     0
     1     2     1     0     0     0
     1     3     3     1     0     0
     1     4     6     4     1     0
     1     5    10    10     5     1
```

3.

```
% Problem 13
% Calculate the reciprocal Fibonacci constant ψ for given n
clc; clear;
n = input('Enter the number of Fibonacci terms (e.g., 10, 50, 100): ');
F = zeros(1, n);
F(1:2) = [1 1];
for i = 3:n
    F(i) = F(i-1) + F(i-2);
end
psi = sum(1 ./ F);
fprintf('The reciprocal Fibonacci constant ψ for n = %d is %.10f\n', n, psi);
```

```
Enter the number of Fibonacci terms (e.g., 10, 50, 100): 10
The reciprocal Fibonacci constant ψ for n = 10 is 3.3304690408
```

```
Enter the number of Fibonacci terms (e.g., 10, 50, 100): 50
The reciprocal Fibonacci constant ψ for n = 50 is 3.3598856661
```

```
Enter the number of Fibonacci terms (e.g., 10, 50, 100): 100
The reciprocal Fibonacci constant ψ for n = 100 is 3.3598856662
```

4.

```
% Problem 15
% Estimate pi using the iterative nested square root formula
clc; clear;
format long;
N = input('Enter the number of terms: ');
prod = 1;
term = sqrt(2);
for k = 1:N
    prod = prod * (term / 2);
    term = sqrt(2 + term);
end
pi_est = 2 / prod;
fprintf('Estimated pi with %d terms = %.15f\n', N, pi_est);
fprintf('Actual pi = %.15f\n', pi);
```

```
Enter the number of terms: 5
Estimated pi with 5 terms = 3.140331156954753
Actual pi = 3.141592653589793
Enter the number of terms: 10
Estimated pi with 10 terms = 3.141591421511200
Actual pi = 3.141592653589793
Enter the number of terms: 40
Estimated pi with 40 terms = 3.141592653589794
Actual pi = 3.141592653589793
```

5.

```
% Problem 18
% Find all integer triples (a, b, c) <= 50 satisfying a^2 + b^2 = c^2
clc; clear;
triples = [];
for a = 1:50
    for b = a:50
        c = sqrt(a^2 + b^2);
        if c == floor(c) && c <= 50
            triples = [triples; a, b, c];
        end
    end
end
disp(' Pythagorean triples (a, b, c) ≤ 50:')
disp(triples)
```

```
 Pythagorean triples (a, b, c) ≤ 50:
     3     4     5
     5    12    13
     6     8    10
     7    24    25
     8    15    17
     9    12    15
     9    40    41
    10    24    26
    12    16    20
    12    35    37
    14    48    50
    15    20    25
    15    36    39
    16    30    34
    18    24    30
    20    21    29
    21    28    35
    24    32    40
    27    36    45
    30    40    50
```

6.

```
% Problem 25
% Compute a^x using the Taylor series expansion
clc; clear;
a = input('Enter base a: ');
x = input('Enter exponent x: ');
% Initialize
n = 0; Sn = 0; E = 1;
while E > 1e-6
    cn = ((log(a))^n * x^n) / factorial(n);
    Sn_new = Sn + cn;
    if n > 0
        E = abs((Sn_new - Sn) / Sn);
    end
    Sn = Sn_new;
    n = n + 1;
end
fprintf('Using Taylor series: %.8f\n', Sn);
fprintf('Using MATLAB built-in: %.8f\n', a^x);
```

```
Enter base a: 2
Enter exponent x: 3.5
Using Taylor series: 11.31370796
Using MATLAB built-in: 11.31370850
Enter base a: 6.3
Enter exponent x: 1.7
Using Taylor series: 22.84961255
Using MATLAB built-in: 22.84961748
```

7. Tracing Exercise

```
n = [1 -88 300 27 -10 17];
p = n(1);
for j = 3:length(n),
p = p + n(j)
end
```

Output:
300+1 -> P = 301
301+27 -> P = 328
328-10 -> P = 318
318+17 -> P = 335

```
k = 0;
tt = 10;
while k < 15
k = k + 3
tt = tt + k
end
disp (k)
disp (tt)
```

Output:

1st Loop	2nd Loop	3rd Loop	4th Loop	5th loop
0+3 -> K = 3	3+3 -> K = 6	6+3 -> K = 9	9+3 -> K = 12	12+3 -> K = 15
10+3 -> tt = 13	13+6 -> tt = 19	19+9 -> tt = 28	28+12 -> tt = 40	40+15->tt = 55

15
55

References:

Gilat, A. (2017). MATLAB: An introduction with applications (6th ed.). John Wiley & Sons.